FPT Education

TRƯỜNG ĐẠI HỌC FPT

Module 1 : Introduction to Flutter

Contents

- Overview of Flutter
  - What is Flutter - Why Flutter
  - Flutter architecture ?
- Dart Language Basics
  - Introduction to Dart and its role in Flutter
- Development Environments
  - DartPad, VS Code, Android Studio
- Setting Up Flutter
  - Installing SDK & Android Studio
  - Configuring Emulator
- Creating & Running Your First App

What is Flutter?

- Flutter is a UI toolkit developed by Google for building natively compiled apps from a single codebase.
- Supports Android, iOS, Web, macOS, Linux, and Windows.
- Written in Dart using its own rendering engine (Skia).
- Enables rapid UI development through reusable widgets.

Why Flutter?

Feature	Description
Fast Development	Hot Reload / Hot Restart
Beautiful UI	Material & Cupertino widgets
Single Codebase	Write once → deploy everywhere
Performance	Nearly native speed

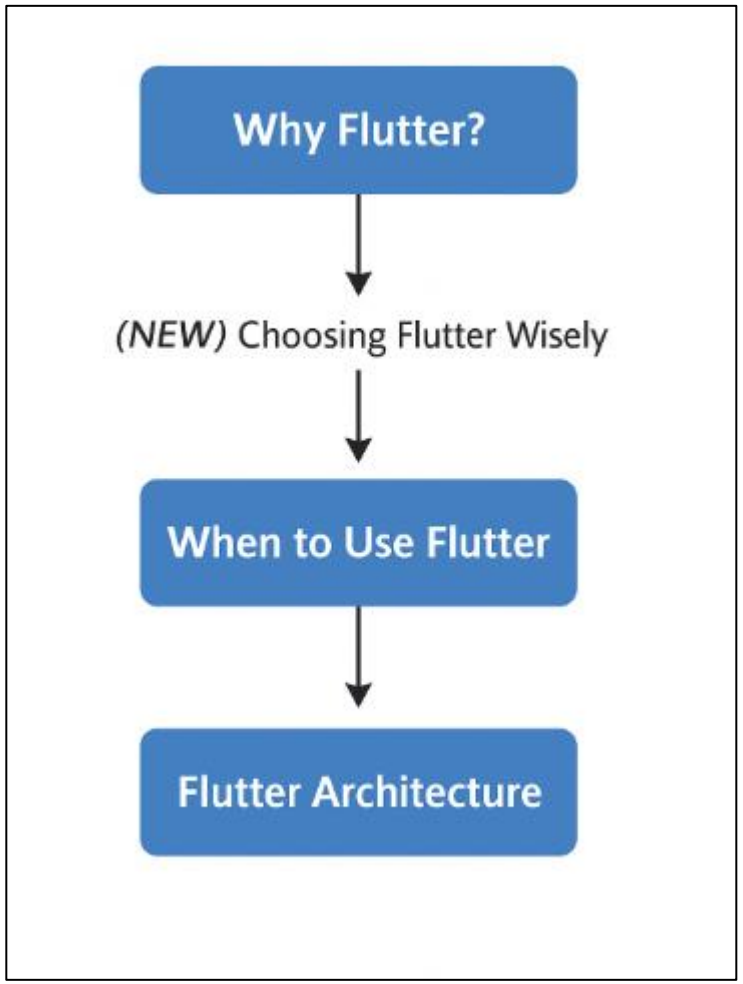
Choosing Flutter Wisely

From Features to Project Decisions

- Flutter offers many powerful features
- However, no technology fits all projects
- Choosing the right tool depends on project context

Key message:

- *Understanding when to use Flutter is as important as knowing why Flutter is powerful.*



When Is Flutter a Good Choice?

- Cross-platform applications with shared UI and logic
- MVPs, startups, and rapid prototyping
- Business applications (e-commerce, booking, dashboards)
- Apps with moderate use of device features
- Teams with limited native Android/iOS resources
- *Flutter is most effective when the goal is time to market, reduced development costs, and maintaining a single codebase.*

Flutter with Native Integration

Content:

- Flutter handles UI and business logic
- Native code handles platform-specific features
- Communication via Platform Channels

Examples of native integration:

- Camera, Bluetooth, NFC
- Payment SDKs
- OS-level services

Flutter doesn't eliminate native code, but integrates with it.

- When Flutter doesn't directly support a feature,
- we can write native Android code (Kotlin/Java)
- or native iOS code (Swift) and connect via Platform Channels.
- This is a very common model in real-world projects.

When Native Development Is Preferable

- Applications requiring deep OS customization
- High-performance games or real-time graphics
- Very low-level hardware access
- Strict platform-specific UI/UX requirements

Key takeaway:

- Flutter is powerful, but not always the optimal choice.

A Brief History

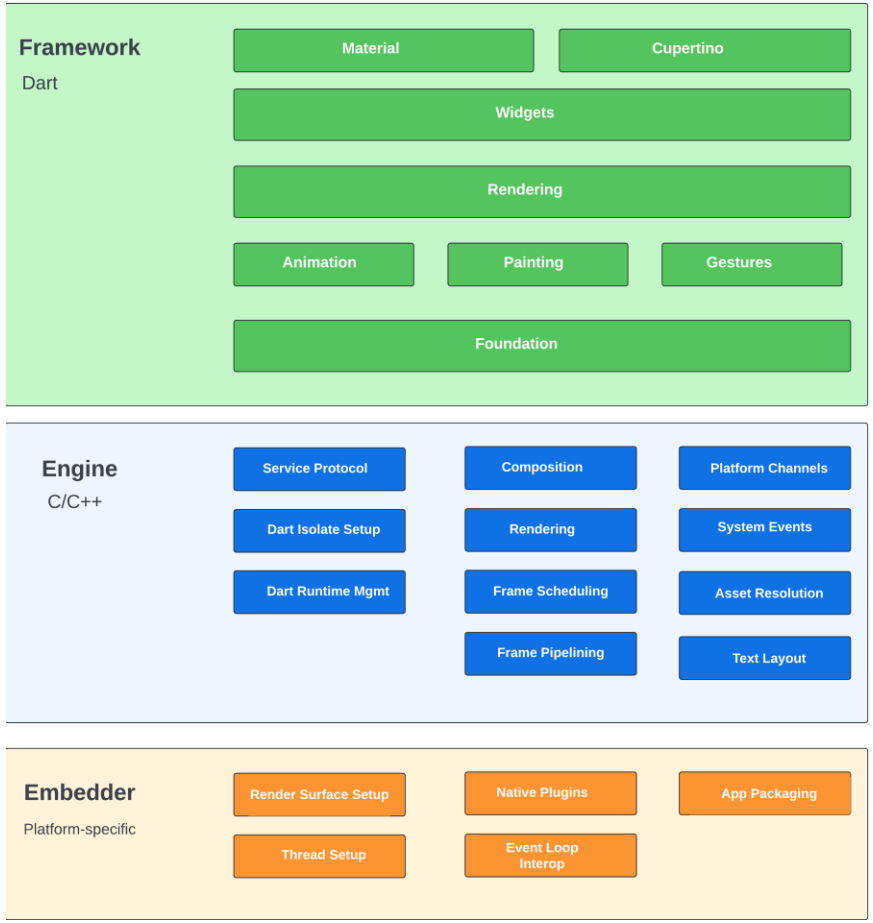
- 2015: Sky project.
- 2017: Flutter Alpha.
- 2018: Flutter 1.0.
- 2021: Flutter 2.0 adds Web/Desktop.
- 2023–2025: Flutter 3.x stable.

Flutter Architecture Overview

Three Layers:

- 1. Framework (Dart)
- 2. Engine (C++)
- 3. Embedder

Workflow: Widget Tree → Render Tree → Composited Scene → GPU Rendering



Framework Layer

- Widgets (Stateless, Stateful)
- Rendering and Layout system
- Foundation libraries
- Key concept: Everything in Flutter is a Widget.

Engine Layer - Embedder Layer

Engine Layer:

- Written in C++, powered by Skia.
- Handles rendering, accessibility, text layout.
- GPU acceleration ensures consistency.

Embedder Layer:

- Platform-specific integration.
- Handles input, lifecycle, window management.
- Custom embedders for Raspberry Pi.

Benefits of Flutter

- Performance: Compiles to ARM code.
- Consistency: Same UI across devices.
- Flexibility: Rich widget set.
- Flutter vs Others

Framework	Language	Output
Flutter	Dart	Native UI + Engine
React Native	JS	Bridges to Native
Native (Android/iOS)	Kotlin/Swift	Pure Native

Dart: Flutter's Programming Language

- Designed for UI creation.
- Combines AOT and JIT compilation.
- Supports null safety and strong typing.

Example:

```
void main() {
    print('Welcome to Flutter!');
}
```

Dart Core Features

- Object-Oriented.
- Functions as first-class objects.
- Null safety.
- Async/Await.

Flutter Development Environments Overview

- Flutter can be developed in three main environments, depending on the learner's goal and hardware setup:

Environment	Description	Ideal For
DartPad	Online editor, runs Flutter code in a browser	Quick demos, learning syntax
VS Code	Lightweight IDE with Flutter & Dart extensions	Fast prototyping, students
Android Studio	Full IDE with SDK, emulator, profiler	Complete mobile app projects

- Start students with DartPad → then move to VS Code / Android Studio.
- Emphasize that all tools compile to the same Flutter runtime.

Using DartPad (Online Environment)

- **URL: https://dartpad.dev**
- **Key Features**
  - No installation required
  - Run Flutter code instantly in browser
  - Supports core widgets (Scaffold, Text, Column, etc.)
  - Ideal for live demo and exercises

Installing the Flutter SDK

- Download from flutter.dev
- Extract the folder (e.g., C:\src\flutter or /Users/you/flutter)
- Add Flutter to the system PATH
- Verify installation:

```
flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.19.6, on macOS 14.2.1 23C71 darwin-arm64, locale en-US)
[✓] Android toolchain - develop for Android devices (Android SDK version 34.0.0)
[✓] Xcode - develop for iOS and macOS (Xcode 15.1)
[✓] Chrome - develop for the web
[✓] Android Studio (version 2022.3)
[✓] VS Code (version 1.88.1)
[✓] Connected device (4 available)
[✓] Network resources
```

Figure 1.2: Flutter doctor

Installing Android Studio

Purpose: Android Studio provides IDE, SDK tools, and emulator to build and run Flutter apps.

Steps:

- Download and install Android Studio
- Install Flutter Plugin
  - Go to Preferences → Plugins → Flutter
  - Dart plugin installs automatically

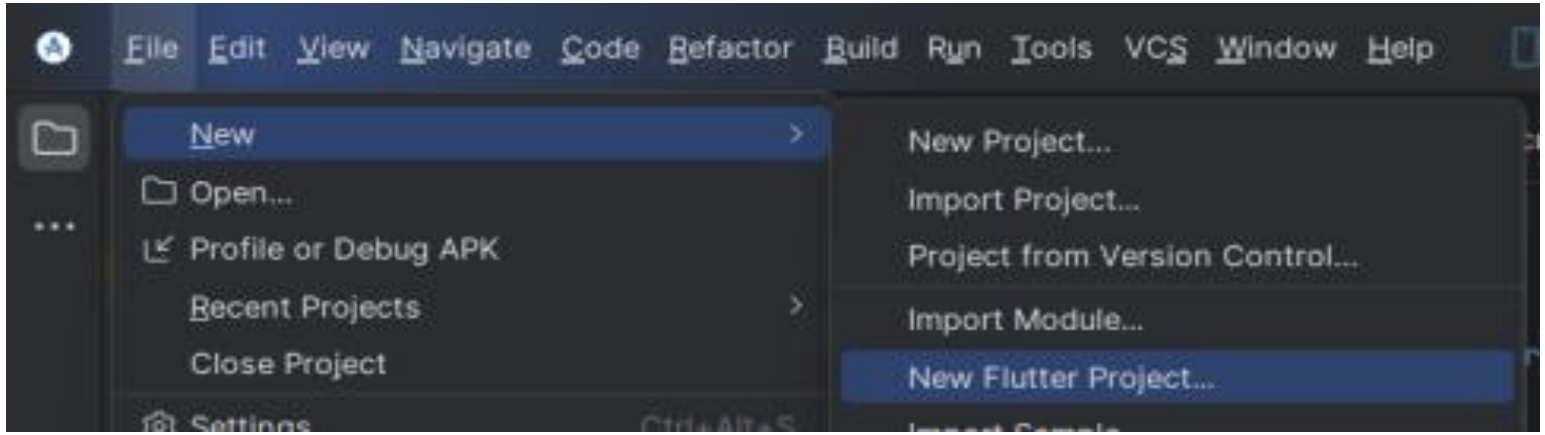
Note: Ensure Android SDK and command-line tools are installed.

Android Studio Interface Overview

Section	Description
Project View	Manage project folders and files
Editor	Write and edit Dart code
Run / Debug Toolbar	Build, run, and debug
Terminal	Access Flutter CLI commands
Device Manager	Manage emulators and devices
Logcat	View Android logs and debugging info

Create a New Flutter Project

- Click File → New → New Flutter Project
- Select Flutter SDK path
- Enter project name: movies
- Organization ID: com.fptu.movies
- Choose platforms: ✅ Android ✅ iOS ✅ Web ✅ macOS ✅ Windows



Flutter Project Structure

Folder	Description
lib/	Contains main Dart code
android/	Android platform-specific code
ios/	iOS platform-specific code
web/	Web app files
pubspec.yaml	Dependencies and assets configuration
test/	Unit test files

`lib/main.dart` **is the app entry point.**

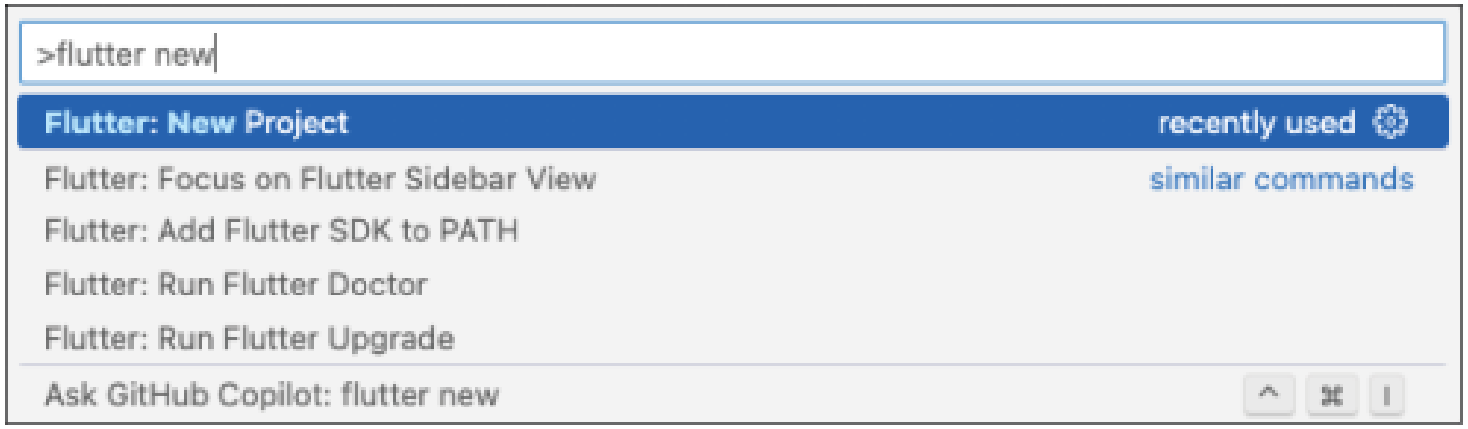
Launching an Emulator

To create an emulator:

- Open Device Manager → Create Device
- Choose a Pixel model (e.g., Pixel 8)
- Select API 33+ (Android 13)
- Set Internal Storage: 4GB
- Note: Use Pixel 3a / API 30 for low-spec PCs.

Using VS Code (Lightweight IDE)

- **Setup Steps**
  - Install VS Code
  - Add Flutter & Dart extensions
  - Open a folder → Ctrl + Shift + P → Flutter: New Project
  - Run the app with F5 or terminal command flutter run



Your First Flutter App ("Hello World")

Goal: Show how one simple Flutter program runs identically in all 3 development environments.

Run the App:

Environment	Steps
DartPad	Open dartpad.dev → New → Flutter → Paste code → ▶ Run
VS Code	Ctrl + Shift + P → Flutter: New Project → Replace lib/main.dart → flutter run or **F5**
Android Studio	*New Flutter Project* → Paste code → ▶ Run on emulator

Summary

- Understand what Flutter is and how it evolved.
- Describe the three-layer architecture: Framework → Engine → Embedder.
- Recognize the benefits of Flutter vs other frameworks.
- Understand Dart basics and its role in Flutter.
- Set up the development environments: DartPad, VS Code, Android Studio.
- Install Flutter SDK and verify setup using flutter doctor.
- Create and run your first app – “Hello World” – on three platforms.

References

- Mastering Flutter 2025 – Ch.1
- docs.flutter.dev
- **https://dartpad.dev**
- dart.dev/guides
- github.com/flutter/flutter